

PROGRAMMAZIONE AD OGGETTI VS SPAGHETTI CODE->

Come già sappiamo la programmazione ad oggetti è un tipo di programmazione che prevede ordine e disciplina. Si parla soprattutto dei metodi del nostro programma che devono avere solamente un compito. Ad esempio un metodo che si chiama calcolaDanni() deve semplicemente fare il calcolo matematico, e non deve portarsi appresso altre operazioni inutili, così che tale funzione sia richiamabile ovunque. Quello che distingue la programmazione "Spaghetti code" da quella ad oggetti è proprio il tipo di programmazione che si svolge. Lo Spaghetti code si porta dietro metodi chilometrici che usano variabili globali, non riutilizzano il codice già scritto, ma soprattutto gli oggetti non parlano tra loro, e così si passa alla programmazione procedurale che si usa in c. Una cosa importante per avere una programmazione pulita che non sia Spaghetti code, è quella di nominare i metodi in un nome che descriva L'UNICA operazione che svolgono.

COMMAND/QUERY SEPARATION->

Per capire se stai programmando bene, si devono separare i metodi del programma in 2 categorie diverse:

Query(domanda)-> Restituiscono un informazione che non modifica nessun dato;

Command(l'ordine)-> Modificano lo stato del sistema, ma non restituiscono nulla(void).

RIUSO DEL CODICE->

Il codice del nostro software per essere un buon codice con programmazione ad oggetti deve essere riutilizzato, e quindi significa non creare codice da 0. Ci sono 2 tipi di riutilizzi del codice:

-Riuso a white-box-> Immaginalo come una scatola di vetro dove all'interno si vedono tutti gli ingranaggi. Quindi riusi il codice avendone visibilità completa, e spesso vai a modificare o estendere il comportamento intimo. Un esempio è l'estensione di una classe, e la classe figlia tramite l'override riutilizza i metodi della classe madre alterandoli. Il vantaggio è che è un approccio rapido e flessibile, lo svantaggio è che se andremo a modificare la classe madre, quella figlia potrebbe crollare.

-Riuso a black box-> Qui invece abbiamo una scatola in metallo chiusa dall'esterno dove all'interno non vedi niente. Un esempio è usare le librerie classiche di java come ArrayList o String, e quando chiami ad esempio length, ci stiamo fidando del codice sotto a quei metodi contenuti nelle librerie. Il vantaggio è che preserva l'incapsulamento in modo assoluto, ma lo svantaggio è che è meno flessibile.

TESTING DEL CODICE->

Il testing del codice è una cosa abbastanza utile nella programmazione in java, per verificare che tutto sia corretto. Il test prevede 3 valori, un valore in input, un valore atteso e il risultato effettivo a run-time. In particolare se il valore atteso ed il valore effettivo combaciano allora il test è verde ed è andato a buon fine, altrimenti è rosso e si deve sistemare il codice. I test

devono essere svolti in ambiente totalmente controllato, ossia che ad essere testato sia solamente un metodo, così che se fallisce sappiamo che l'errore arriva da quel metodo. Tramite la parola chiave `@Test` sopra le classi o i metodi che vogliamo testare, sarà J-unit ad unire tutti loro e testarli in un unico colpo. Quando eseguiamo un test il TDD(test driven development) consiste nello scrivere prima il test e poi il codice che risolve il requisito. Consiste nello scrivere codice solo se nel nuovo test ci sono errori. Questo ci viene comunicato tramite dei codici:

RED-> Il test non è superato, quindi si può andare a ritoccare il codice per far andare a buon fine il test;

GREEN-> Test superato, quindi o si fanno altri test o si passa al refactoring;

REFACTORING-> Si migliora la struttura del codice.

Anche qui ci sono 2 tipi di test:

-**Test white-box**: Questo test lo fa chi conosce veramente il codice, e questo significa che sa cosa c'è esattamente dentro il metodo. Testa se tutte le righe di codice vengano eseguite(copertura del codice);

-**Test black-box**: Qui invece abbiamo un programmatore a cui non interessa come è scritto il codice e non lo conosce, ma deve solo scrivere il codice del test e verificare che i 2 output combacino. Un bravo programmatore pensa fuori dagli schemi dato che il codice non è suo, quindi potrebbe rilevare altre anomalie.

Ogni test ha dei principi che deve rispettare per essere un buon test:

FIRST-> Sono un gruppo di principi fondamentali che un test deve avere:

-**Fast**: Deve essere veloce, se un test ci impiega tanto a svolgersi non è un buon test;

-**Isolato/indipendente**: Un test deve dipendere solo da se stesso, quindi se fallisce fallisce solo lui;

-**Ripetibile**: Un test deve essere ripetibile;

-**Self-validating**: Deve essere il test a dirci se ha avuto successo o ha fallito, e non deve essere il programmatore ad andarlo a leggere;

-**Timely**: I test devono essere scritti sempre al momento giusto;

-J-UNIT->

j-unit è un framework usato per eseguire test su classi e metodi, quindi è la pratica del TDD. Le chiavi principali su come usare j-unit sono:

-**Annotazioni**: Le annotazioni sono quelle messe sopra le classi o i metodi per capire cosa il compilatore deve fare con loro, quindi nel caso dei test per una classe metteremo sopra la classe la parola `@Test`. oppure quando c'è una funzione di una classe figlia che eredita dalla madre metteremo `@Override`;

-**Asserzioni**: è il vero cuore del test perché sono quelli che permettono di verificare tramite il test se quella classe fa il suo lavoro o no. Sono comandi come (AssertEquals, AssertTrue, AssertFalse, AssertNull);

-**Test-runner**: Con un solo comando fa partire tutti i test di j-unit.

INFO AGGIUNTIVE SUI TEST->

La fase di controllo si divide in due domande diverse che il professore farà all'orale:

- **Verifica (Solitamente White-box)**: Risponde alla domanda "*Stiamo costruendo il sistema nel modo corretto?*". Controlla la qualità del codice e se rispetta il design.
- **Validazione (Solitamente Black-box)**: Risponde alla domanda "*Stiamo costruendo il sistema corretto?*". Controlla se il prodotto soddisfa i reali bisogni e le richieste dell'utente finale.
- I bug si dividono in tre categorie in base a dove nascono: **Difetti di specifica** (requisiti ambigui), **Difetti di progettazione** (architettura o UML sbagliati), **Difetti di codice** (errori di implementazione in Java). Quando risolvi un bug o aggiungi una funzione, rischi spesso di rompere pezzi di codice che prima funzionavano. I test devono essere **ripetibili e riusabili** proprio per questo: ogni volta che tocchi il codice, fai ripartire tutti i vecchi test (Test Runner di JUnit) per assicurarti che non ci siano "regressioni" (ossia passi indietro nella qualità).
- **Regola aurea**: Il testing può solo dimostrare la *presenza* di errori, **mai la loro assenza**. Un test esaustivo (totale) è impossibile nella realtà.

Test di Regressione

Quando risolvi un bug o aggiungi una funzione, rischi spesso di rompere pezzi di codice che prima funzionavano.

- I test devono essere **ripetibili e riusabili** proprio per questo: ogni volta che tocchi il codice, fai ripartire tutti i vecchi test (Test Runner di JUnit) per assicurarti che non ci siano "regressioni" (ossia passi indietro nella qualità).

Quando facciamo un test su una funzione che controlla range di numeri, ovviamente nelle asserzioni andremo a mettere un range determinato di numeri, e non tanti test sui singoli numeri, sennò impazziremo.

REFACTORING DEL CODICE->

Il refactoring del codice è una tecnica che non serve a cambiare i risultati del codice, ma serve a migliorare il codice cambiando la struttura mano a mano che il codice va ad evolversi. Quindi consiste nella manutenzione del codice già esistente che porta da un codice che soddisfa i requisiti, ad uno che soddisfa i requisiti ma con una struttura migliore. In generale il refactoring riduce la dimensione del codice e il debito tecnico del progetto, cioè che quando dovrò correggere il codice in futuro se più corto tramite il refactoring ci starò meno tempo. Il refactoring aiuta nella ricerca dei bug perché si rivede il codice. Ovviamente

facendo refactoring si deve controllare che non abbia alterato la correttezza del codice. Ci sono vari modi di fare refactoring:

-ESTRAI METODO:

In parole povere la tecnica estrai metodo dallo stesso titolo, prende quelli che sono metodi troppo lunghi, ne taglia un pezzo che fa uno specifico lavoro, lo estrae da quel metodo e crea un metodo nuovo inserendogli quel codice e dandogli un nome preciso per descrivere quello che fa.

```
public void stampaDettagli(){
    this.stampaNome();
    System.out.println("Prezzo: 30 euro"); // Hard coded, possibili
    ripetizioni
}

// Diventa...
public void stampaDettagli(){
    this.stampaNome();
    this.stampaPrezzo(); // Con interfaccia riutilizzabile
}
```

Ad esempio qui vediamo che prima abbiamo un sout che stampa il prezzo, e ci potrebbero essere degli hard coded, ossia che dato che questa stampa viene fatta in più punti se dovessimo cambiare il prezzo dovremmo andare a cercare tutti i punti in cui è scritto e cambiarli tutti. Invece con la chiamata del metodo lo cambieremo in un solo punto. Dopo aver applicato la tecnica il metodo stampaDettagli() è diventato pulitissimo e chiarissimo e chiunque capisce cosa fa.

Per gestire le variabili abbiamo 2 casi:

Se quella variabile nel metodo vecchio era una variabile presa in input nel metodo e quella parte di metodo che tagliamo la usa, allora il nuovo metodo diventa: void stampaPrezzo(int variabile). Se invece era una variabile temporanea basta ricrearla nell'altro metodo.

-SOSTITUISCI TEMP CON QUERY:

Questa tecnica di refactoring è la naturale evoluzione di 2 concetti appena visti, la query(restituisce un valore senza modificare nulla) e la tecnica estrai metodo.

Di solito quando scriviamo dei metodi, all'interno creiamo delle variabili temporanee per salvare i risultati di determinate operazioni. Negli altri metodi non possiamo usare tali variabili perché sono temporanee, quindi saremo costretti a ricreare queste variabili anche negli altri metodi.

LA SOLUZIONE è quella che invece che salvare la formula matematica in una variabile, crei un metodo apposta per quella formula, e quando ti serve chiami il metodo così che hai la formula pronta. Infatti tramite il nome della tecnica da temp a query si capisce subito perché viene chiamata così. L'esempio con un codice è questo:

```
double prezzo = quantita * prezzo;

// Diventa una chiamata alla funzione ...
private double getPrezzo(){
    return quantita * prezzo;
}
```

Abbiamo sopra la temp ossia la variabile temporanea prezzo, e poi calcoliamo tale prezzo. Sotto invece lo definiamo dentro la funzione, e così quando ci servirà lo avremo pronto senza dichiarare variabili.

-DIVIDI VARIABLE TEMP:

La variabile temporanea che abbiamo spesso usato nei cicli for in C, come sappiamo soprattutto nei cicli for o nei while cambia sempre di significato, e questo significa che un programmatore non può mai ricordarsi tutti i valori che quella variabile ha avuto nel corso dell'esecuzione del programma. Lo scopo di utilizzo di queste variabili temporanee è ovviamente il risparmio di memoria, ma così facendo tutti i valori di temp non si riconosceranno mai. La soluzione è quella di trascurare l'occupazione della memoria e fare una variabile per ogni tipo di cosa che ci serve.

```
double temp = 2*(height+width); // temp è il perimetro del rettangolo
temp = height*width; // Adesso temp è l'area del rettangolo

// Per avere nomi più appropriati ed avere un codice più leggibile diventa
...
final double perimeter = 2*(height+width);
final double area = height*width;
```

Con il final abbiamo garantito che quelle 2 variabili non possano più essere riassegnate a nessun altro tipo di valore. Quindi se ci sarà un'altra variabile chiamata area ci sarà un errore.

-ESTRAI CLASSE:

Quando si fa refactoring, la tecnica dell'estrai classe è la più importante, perché si riferisce a quelle classi che fanno troppe cose e fanno troppe cose. Una classe dovrebbe avere un solo concetto da gestire, se ne ha 2 è il momento di estrarre.

```
Java

// CLASSE MONOLITE (Prima del refactoring)
class Pilota {
    private String nome;
    private String squadra;

    // Campi che non c'entrano strettamente con la guida!
    private String via;
    private String citta;
    private String codicePostale;

    public String getIndirizzoCompleto() {
        return via + ", " + citta + " " + codicePostale;
    }
}
```

Questa è la classe prima del refactoring, e come vediamo la classe si chiama pilota, ma al suo interno ha variabili che non c'entrano assolutamente niente con lui e con le sue abilità di

guida.

```
Java

// 1. Nuova classe estratta che ha una sola responsabilità
class Indirizzo {
    private String via;
    private String citta;
    private String codicePostale;

    public Indirizzo(String via, String citta, String codicePostale) {
        this.via = via;
        this.citta = citta;
        this.codicePostale = codicePostale;
    }

    public String getCompleto() {
        return via + ", " + citta + " " + codicePostale;
    }
}

// 2. La classe originale alleggerita (Riuso a Black-Box!)
class Pilota {
    private String nome;
    private String squadra;

    // Uso la composizione: il pilota HA UN indirizzo
    private Indirizzo residenza;

    public Pilota(String nome, String squadra, Indirizzo residenza) {
        this.nome = nome;
        this.squadra = squadra;
        this.residenza = residenza;
    }
}
```

Come vediamo qua abbiamo preso tutti gli attributi e metodi che non c'entravano con il pilota e abbiamo creato una classe a parte solo per gli indirizzi e tutto il resto, quindi abbiamo estratto dalla classe per creare un'altra classe, proprio come l'estrarre un metodo. Questa tecnica si può applicare quando:

- **Sotto-gruppi di dati:** Noti 3 o 4 variabili che hanno sempre lo stesso prefisso (es. tempOlio, tempAcqua, tempScarico ... potresti estrarre una classe TemperatureMotore).
- **Metodi isolati:** Ci sono metodi che usano solo quel sotto-gruppo di variabili e non toccano il resto della classe.
- **Cambiamenti disaccoppiati:** Ti rendi conto che se cambia una logica (es. come si formatta un testo o un indirizzo), devi toccare una classe che fa tutt'altro (il Pilota o l'Auto).

-SPOSTA METODO:

Questa tecnica del refactoring si usa quando un metodo ha l'invidia delle caratteristiche, ossia che se un metodo si trova dentro la classe A, ma usa continuamente metodi della classe B, quindi anche gli attributi. Significa che ha sbagliato "casa" e logica vuole che debba stare nella classe dove usa di più gli attributi.

```

Java

// PRIMA DEL REFACTORING: Il metodo è "invidioso" della classe Moto
class Pilota {
    private String nome;

    // Questo metodo è dentro Pilota, ma usa SOLO i dati della Moto!
    public double calcolaAssicurazioneGara(Moto moto) {
        if (moto.getCilindrata() > 600) {
            return moto.getValoreDiMercato() * 0.05 + 200;
        } else {
            return moto.getValoreDiMercato() * 0.03 + 100;
        }
    }
}

```

Prima dell'applicazione del metodo vediamo come il metodo `calcolaAssicurazioneGara` utilizzi attributi e metodi della classe `Moto`.

```

Java

class Moto {
    private int cilindrata;
    private double valoreDiMercato;

    // Passo 1: Il metodo ora è a casa sua, non serve più passare la moto come parametro!
    public double calcolaAssicurazioneGara() {
        if (this.cilindrata > 600) {
            return this.valoreDiMercato * 0.05 + 200;
        } else {
            return this.valoreDiMercato * 0.03 + 100;
        }
    }

    // Getter...
}

class Pilota {
    private String nome;

    // Passo 2: Il vecchio metodo delega semplicemente il lavoro alla Moto
    public double calcolaAssicurazioneGara(Moto moto) {
        return moto.calcolaAssicurazioneGara();
    }
}

```

Adesso che abbiamo spostato quel metodo nella classe dove è giusto che sia, nella classe `Pilota` andremo semplicemente a chiamare il metodo sempre con un'istanza di `Moto`.

PROCESSI DI SVILUPPO SOFTWARE->

CARATTERISTICHE DEL SOFTWARE:

Come già enunciato prima il software è modificabile e può subire variazioni di vario tipo, grazie al fatto che non possiede parti fisiche, e si può adattare a qualsiasi tipo di hardware. Questi cambiamenti sono dati dal successo che un software ha. Se un software ha tanto successo ha ovviamente bisogno di cambiamenti (aggiornamenti) proprio come in un videogioco di successo.

QUALITÀ DEL SOFTWARE->

Le qualità del software vengono indicate da diverse caratteristiche, che sono:

CORRETTEZZA-> La correttezza del software indica se un software venduto ad un cliente svolge realmente quello che dovrebbe fare, e quindi è corretto;

EFFICIENZA->Nessuno spreco inutile di risorse;

MANUTENIBILITÀ->Facilità ad introdurre modifiche;

SICUREZZA->Che si divide in safety: Che è la sicurezza delle persone dalla parte hardware su cui è installato il software; Security: È la capacità di difendere i dati del sistema(Limita gli accessi);

USABILITÀ->Indica quanto è facile il software da usare.

La capacità del software di essere modificato, è già una qualità che un software deve avere a prescindere.

Un software per essere fatto bene deve possedere dei requisiti che possano renderlo ottimale. Per farlo si passa attraverso 5 fasi:

-**Analisi dei requisiti**: Capire cosa serve al cliente per progettargli un buon software;

-**Progettazione**: Disegnare prima dei diagrammi UML per fare una panoramica, e capire quali pattern usare;

-**Implementazione**: Scrivere fisicamente il codice in java;

-**Convalida / Testing**: Verificare con dei test se quello che è scritto è corretto e rispetti le richieste;

-**Manutenzione**: Correggere i bug che escono fuori, fare refactoring, e aggiornare lungo il tempo il programma, perché se ha successo allora si dovranno apportare novità.

ANALISI DEI REQUISITI:

Ci si sofferma di solito sul primo punto, perché l'azienda produttrice di software deve capire se ha veramente le risorse e competenze per fare quel software, deve chiedere al cliente cosa vuole veramente, e poi stilare un accordo chiaro su cosa quel programma dovrà fare. Infine un programma software avrà 2 tipi di requisiti da soddisfare:

-**Funzionali**: Ossia che è qualcosa che deve fare il cliente, come ad esempio inserire la temperatura dell'olio in un sistema che controlla il motore, quindi è qualcosa che deve fare il programma;

-**Non funzionali**: Invece questi requisiti non sono qualcosa che il cliente deve fare tramite il programma, ma sono dei requisiti che il programma deve rispettare, come ad esempio un limite di tempo nell'esecuzione, o l'utilizzo di un determinato componente software per una buona manutenzione.

PROGETTAZIONE:

Questa è la fase dove si prende il documento di come il cliente vuole il suo programma, e si fa una struttura iniziale, quindi un diagramma UML o uno schema. Si divide in:

-**Suddivisione dei requisiti**: Prendiamo le richieste enormi e le spezzettiamo in problemi più piccoli e gestibili;

-**Progettazione dell'architettura**: Decidiamo la struttura "macro" del sistema (es. useremo un'architettura client-server? Useremo un database relazionale?);

-**Specifica delle responsabilità**: Decidiamo quali "moduli" faranno cosa. (es. "Il modulo A si occuperà della grafica, il modulo B dei calcoli fisici").

-**Progettazione di interfacce, componenti, algoritmi**: È il livello "micro". Qui disegniamo letteralmente i diagrammi delle classi, decidiamo di usare un *Factory Method* o uno *Strategy*, e definiamo le firme dei metodi.

CONVALIDA:

Dopo aver scritto il codice si deve convalidare prima di consegnarlo al cliente, e quindi verificare che sia tutto apposto e non ci siano bug. Se si trovassero bug sarebbe un normale processo di sviluppo. Ci sono 4 fasi:

-**Unit Test (Test di Unità)**: È il livello più basso. Testi una singola funzione o una singola classe in totale isolamento;

-**Test di Sistema**: Fai un passo indietro e testi l'intero programma assemblato. Verifichi che i vari moduli comunichino bene tra loro;

-**Alpha Test**: Il software è quasi pronto. Viene testato internamente dagli sviluppatori stessi;

-**Beta Test**: Il collaudo su strada. Dai il software in mano a un gruppo di clienti/utenti veri. Loro lo usano nel mondo reale e ti segnalano i crash o i comportamenti anomali.

Da qui nasce la nomenclatura per: Versione alpha: con difetti e parti mancanti;

Versione beta: usata dai clienti, ha tutte le feature ma con difetti, eventualmente segnalati dagli utilizzatori;

Versione gold: completa e funzionante;

SVILUPPO SOFTWARE->

Lo sviluppo software ci dice come organizzare il lavoro nel tempo quando si deve creare un programma da 0, e quindi su come gestire un progetto. Si divide in 2 grandi "religioni":

SVILUPPO A CASCATA: Lo sviluppo a cascata è l'approccio più antico ma anche il più spietato, perché come l'acqua scorre solo verso il basso e non si può tornare più indietro. Fai l'analisi dei requisiti -> *Finita*. Fai il design -> *Finito*. Scrivi il codice -> *Finito*.

-**PRO**: sono che si ha tanta documentazione e se il codice finale ha rispettato i giusti requisiti sarà un prodotto di alta qualità;

-**CONTRO**: Parli con il cliente la prima volta e poi non lo vedi fino a che non finisci il lavoro, e se per caso c'è da cambiare qualche cosa in corso d'opera costerà moltissimo tempo e risorse;

SVILUPPO EVOLUTIVO E INCREMENTALE: Tale sviluppo è molto differente da quello a cascata, perché fa un passo alla volta perché non sa cosa vuole alla fine. Si distinguono 2 tipi:

-**Sviluppo per esplorazione:** Parti con una idea di base e lo costruisci insieme al cliente avendo il suo feedback. Il pro è che hai sempre un feedback del cliente e puoi cambiarlo quando vuoi. Il contro è che non hai mai un'idea finale del progetto e potresti avere un'architettura disordinata;

-**Sviluppo build and fix:** È lo sviluppo più grezzo e meno utilizzato, dato che il cliente non sa cosa vuole, e i programmatori iniziano a scrivere codice a caso iniziando a correggere i bug e gli errori che man mano escono. Il Build and Fix è la fabbrica dello Spaghetti Code. **la fase di design è inesistente e il codice è di bassa qualità.**

Poi ci sono:

-**Sviluppo incrementale:** Questo sviluppo funziona con la priorità. Si studia il programma e ci si chiede quale funzionalità sono quelle più importanti senza la quale il programma non funziona. Poi man mano si fanno quelle meno importanti.

-**Processo COTS - Components Off-The-Shelf:** In italiano significa letteralmente preso dallo scaffale. Questo sviluppo non vuole scrivere tutte le strutture dati da 0, ma prende riferimento dalle strutture dati di software precedenti o dallo stesso software che si sta sviluppando in quel momento, per dar vita a quelli nuovi;

-**Modello a spirale:** È il più evoluto e più importante dato che ha 4 passaggi fondamentali:

- **Identificazione obiettivi (Primo Quadrante):** Si definisce cosa deve fare questo pezzo di software nel ciclo corrente e quali sono i vincoli.
- **Valutazione dei rischi (Secondo Quadrante - Il più importante!):** Ci si siede a tavolino e si analizza cosa potrebbe andare storto.
- **Produzione e convalida (Terzo Quadrante):** Solo dopo aver superato l'analisi dei rischi si passa a progettare, scrivere il codice e testarlo.
- **Revisione e pianificazione (Quarto Quadrante):** Si guarda il risultato insieme al cliente, si valuta il lavoro fatto e si pianifica il giro di spirale successivo.

SVILUPPO AGILE:

Lo sviluppo agile è più improntato con la collaborazione nel team, e quindi che riguarda più gli individui, ma soprattutto una collaborazione e sviluppo con il cliente, che con qualsiasi osservazione può far migliorare il codice. L'agilità si rivela nel considerare positive le richieste di cambiamento anche in fasi avanzate, fornire release software frequenti. In presenza di persone esperte questo funziona perfettamente, quando vi sono persone alle prime armi funziona meno.

EXTREME PROGRAMMING->

È un metodo agile e radicale, che porta all'estremo le buone pratiche di programmazione.

Serve soprattutto per progetti con requisiti instabili e scadenze stringenti.

Le 12 Pratiche Chiave (Il succo per l'esame)

1. **Gioco di pianificazione:** Requisiti scritti dal cliente su schede sintetiche (**Story Card**), stimate e divise in task ogni due settimane.
2. **Piccole release:** Rilasci continui per avere feedback immediati dal cliente.
3. **Metafore:** Usare un linguaggio semplice e non ultra-tecnico per farsi capire dal cliente.
4. **Testing:** Test continui scritti prima o insieme al codice. I *Test funzionali* validano i requisiti (punto di vista utente), gli *Unit test* validano il codice (sviluppatori).
5. **Refactoring:** Miglioramento continuo del codice preesistente per evitare lo Spaghetti Code.
6. **Pair Programming:** Si programma **in coppia** su un solo PC. Il *Driver* scrive il codice, il *Navigator* controlla la logica generale e pensa ai test. Condivide la conoscenza e azzera i colli di bottiglia.
7. **Cliente in sede:** Il cliente è fisicamente presente nel team per rispondere subito ai dubbi.
8. **Design semplice:** Scrivere solo il codice che serve oggi, senza anticipare il futuro. Si usano schede **CRC** (Class-Responsibility-Collaboration) per mappare le classi al posto di UML complessi.
9. **Codice posseduto dal team:** Non esiste il "mio" codice. Chiunque può e deve modificare qualsiasi classe per risolvere un problema.
10. **Integrazione continua:** Il codice viene unito al sistema principale ogni poche ore. Se gli Unit Test falliscono, si scarta il codice o si fixa subito.
11. **Settimana da 40 ore:** Niente straordinari folli. Un team stanco lavora male e introduce bug.
12. **Usare standard per i codici:** Stesse regole di formattazione, naming parlante e convenzioni per tutto il team.

SCRUM->

È una struttura base dello sviluppo software che aiuta le persone a generare una soluzione per problemi complessi. L'ambiente è composto da:

-**Product Owner:** elenca il lavoro da svolgere mediante un Product Backlog (elenco dei lavori arretrati);

-**Scrum Team:** ovvero gli sviluppatori, che ha l'obiettivo di trasformare parte di lavoro in un incremento di risultati durante un'iterazione detta Sprint.

Tale framework si basa sull'empirismo, ossia sulle decisioni prese in base ad un'esperienza reale e non su piani teorici, e sul lean, ossia concentrarsi solo sull'essenziale per evitare sprechi di tempo.

I pilastri che formano lo SCRUM sono:

-**Trasparenza**: Tutto il lavoro deve essere visibile a tutti(il team) tranne i documenti chiave;

-**Ispezione**: Controllare sempre il lavoro per trovare i problemi;

-**Adattamento**: Se qualcosa non funziona, si cambia subito rotta;

Gli eventi SCRUM che decidono il lavoro sono 5:

- **Sprint**: Il ciclo di lavoro effettivo (durata fissa: da 2 settimane a 1 mese) in cui si crea un pezzo di software funzionante.
- **Sprint Planning**: Riunione iniziale per decidere cosa fare nello Sprint corrente.
- **Daily Scrum**: Il punto della situazione quotidiano (15 minuti) per ispezionare il progresso.
- **Sprint Review**: Riunione a fine Sprint per mostrare il software al cliente e raccogliere feedback.
- **Sprint Retrospective**: Analisi interna del team su *come* si è lavorato per migliorare i processi nel prossimo ciclo.

EVOLUZIONI E MANUTENZIONE->

La manutenzione è la fase più costosa dello sviluppo software, dato che quando il codice è finito e si è consegnato, non è ancora finito il lavoro, ma c'è da fare la manutenzione che si divide in:

- **Correttiva**: Rimuovere bug e difetti scoperti dagli utenti.
- **Adattiva**: Adeguare il software a nuovi ambienti (es. aggiornamento del sistema operativo).
- **Perfettiva**: Aggiungere nuove funzionalità o migliorare quelle esistenti su richiesta del cliente.
- **Preventiva**: Pulire il codice (es. fare il *refactoring*) per prevenire problemi futuri.

LEGGI DI LEHMAN->

Spiegano come e perché i grandi software si degradano nel tempo se non vengono aggiornati e gestiti. Le leggi sono:

- **Cambiamento continuo**: Un software deve essere modificato costantemente per rimanere utile, altrimenti diventa obsoleto.
- **Aumento della complessità**: Man mano che il software evolve, la sua struttura diventa sempre più disordinata (Spaghetti Code), a meno che non si faccia manutenzione preventiva (refactoring).
- **Continua crescita**: Il programma deve espandere continuamente le proprie funzionalità per mantenere soddisfatto l'utente.
- **Diminuzione della qualità**: Se non viene curata la manutenzione a dovere, la qualità percepita e reale del sistema cala inevitabilmente con il passare del tempo.

Per fare tale manutenzione e aggiornamento, come abbiamo già visto utilizziamo la tecnica del refactoring.

METRICHE->

Le metriche sono delle misure che servono per misurare la qualità, la complessità e la dimensione del codice mentre lo si costruisce.

Ad esempio c'è la complessità ciclomatica(cc), che serve per calcolare il numero di test che servono per ricoprire tutto il codice, e sono:

$$cc = \text{archi} - \text{nodi} + 2$$

Poi c'è la dimensione che si calcola con le linee di codice totali(LOC) diviso gli spazi vuoti e commenti NCNB:

$$LOC/NCNB$$

Metriche CK (Chidamber & Kemerer) per Sistemi ad Oggetti

Servono a valutare nello specifico la complessità e la modularità delle classi:

- **WMC (Weighted Methods per Class):** Somma delle complessità dei metodi di una classe. Valori alti aumentano il lavoro di manutenzione e diminuiscono il riuso (meglio valori bassi).
- **DIT (Depth of Inheritance Tree):** Numero massimo di livelli d'ereditarietà dalla classe alla radice (livello zero). Valori alti indicano più riuso, ma rendono il comportamento delle classi molto difficile da comprendere.
- **NOC (Number of Children of a Class):** Numero di sottoclassi dirette di una classe. Più è alto, più c'è riuso, ma aumenta anche l'influenza (e il rischio d'impatto in caso di modifiche) di quella classe sul resto del sistema.
- **CBO (Coupling Between Object Classes):** Numero di altre classi con cui la classe interagisce. Valori alti significano forte accoppiamento/dipendenza, riducendo riuso e comprensione.
- **RFC (Response for a Class):** Numero di metodi eseguiti in risposta alla ricezione di un messaggio. Valori alti sono sinonimo di alta complessità, rendendo ostici comprensione e testing.
- **LCOM (Lack of Cohesion of Methods):** Misura quanto i metodi di una classe sono slegati tra loro. Valori bassi indicano un'alta coesione, garantendo elevato riuso, semplicità e il rispetto del *Single Responsibility Principle*.

LABORATORIO DI JAVA

Java è un linguaggio imperativo, ma dalla versione 8 di java, è diventata sempre di più funzionale, che è più facile da parallelizzare rispetto a quella ad oggetti. Ad esempio abbiamo l'interfaccia List<> che ci permette di istanziare con la stessa parola tanti tipi di strutture dati che non dobbiamo nemmeno implementare, come array, pile, liste, code.

ESPRESSIONI LAMBDA:

è una funzione anonima che ti permette di passare un blocco di codice come se fosse il parametro di un metodo senza dover creare un'intera classe per farlo.

```
s -> System.out.println("Ciao "+s);  
(x,y) -> x+y; //Questo valore viene tornato  
() -> System.out.println("Buongiorno");  
(x,y) -> { System.out.println("x: "+x); return x+y; } // Devo specificare return
```

Tale espressione lambda si basa sull'operatore freccia(->). A sinistra vanno messi i parametri in ingresso, e a destra va messo il corpo della funzione con eventuale valore restituito.

Quindi `s-> System.out.println("Ciao " +s)` è un'espressione lambda.

Non si è obbligati a scrivere i tipi dei parametri in ingresso, il compilatore ci arriva da solo analizzando il contesto (inferenza). Queste espressioni lambda possono anche NON dipendere solamente dai parametri passati a sx della freccia. Quando il corpo ha più righe di codice si usano le parentesi graffe {}.

Classi anonime: Le classi anonime sono classi senza nomi, definite e istanziate mediante una variante dell'operatore `new`. Sono usate per implementare interfacce.

```
public interface Hello {  
    public void greetings(String s);  
}
```

```
public class Sera {  
    private Hello myh;  
  
    public Sera(Hello h) {  
        myh = h;  
    }  
  
    public void saluti() {  
        myh.greetings("buonasera");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Sera sr = new Sera(new Hello() {  
            public void greetings(String s) {  
                System.out.println("Ciao, " +s);  
            }  
        });  
    }  
}
```

Con la classe anonima sei costretto a scrivere codice ripetitivo, come l'override dell'unico metodo dell'interfaccia, e vediamo come nel costruttore della classe `Sera` dobbiamo andare a metterci `new Hello`.

```
Sera sr = new Sera(s2 -> System.out.println("Ciao, " + s2));
```

Con questa unica riga invece abbiamo risolto tutti i problemi di ripetizione del codice, ma questo si può fare solo se l'interfaccia ha un unico metodo.

```
private List<String> listaNomi = Arrays.asList("Nobita", "Nobi", "Shizuka");
```

Se hai una lista di questo tipo, si può verificare se uno specifico nome è contenuto nella lista tramite 2 modi:

-Imperativo:

```
public void trovaImperativo() {
    boolean trovato = false;

    for (String nome : listaNomi)
        if (nome.equals("Nobi")) {
            trovato = true;
            break;
        }

    if (trovato) System.out.println("Nobi trovato");
    else System.out.println("Nobi non trovato");
}
```

Facciamo un for per scorrere tutta la struttura dati, inserendo anche un'istanza per richiamare gli elementi in lista nomi. Poi con una flag verifichiamo se il nome c'è o non c'è. Qui c'è il codice molto più lungo, e a seguito di una rapida lettura si potrebbero avere difficoltà a capire cosa il codice stia facendo.

-Dichiarativo:

```
public void trovaDichiarativo() {
    if (listaNomi.contains("Nobi"))
        System.out.println("Nobi trovato");
    else
        System.out.println("Nobi non trovato");
}
```

Con la dichiarativa invece abbiamo tutto più chiaro, e con quasi la metà di righe di codice facciamo la stessa cosa, e chiamiamo metodi che sono nella libreria List<> di Java. È più chiaro come metodo, e i cicli vengono fatti all'interno della funzione contains.

PS: Una funzione si dice di ordine superiore, quando come parametro ha un'altra funzione o restituisce una funzione come risultato.

COLLECTION->

È un'interfaccia che definisce il comportamento di tutte le strutture dati, come add(), remove(), size() ecc..

	get	add	contains	remove
ArrayList	costante	costante	$O(n)$	$O(n)$
LinkedList	$O(n)$	costante	$O(n)$	costante
TreeSet		$O(\lg n)$	$O(\lg n)$	$O(\lg n)$

Questi sono i tempi di accesso agli elementi di ogni struttura dati. Abbiamo l'array list che è un array espandibile, e l'accesso al generico elemento avviene in tempo costante. LinkedList è una lista e ogni elemento ha i riferimenti al successivo e al precedente. Vector è simile ad ArrayList ma è sincronizzato nel senso che nel momento in cui ci sono più thread che provano ad accedere agli elementi esso consente l'accesso uno alla volta.

TreeSet sotto al cofano è un albero binario auto bilanciante.

STREAM

Con lo stream entriamo nella programmazione funzionale.

In java uno stream è un flusso di dati (sequenza di elementi), che ti permette di manipolare tali informazioni in maniera dichiarativa, ossia dicendo cosa vuoi ottenere e non come ottenerlo. Devi immaginare gli stream non come un magazzino fisso e fermo (quindi un array o una lista), ma come un nastro trasportatore che prende oggetti da un magazzino, e con delle funzioni intermedie come filter, vengono filtrati tali oggetti. Poi alla fine di tale nastro ci sono operazioni come count che restituiscono il risultato finale. **Uno stream non salva dati al suo interno, non è una struttura dati.** Qualsiasi lista da cui prendi gli elementi e li passi allo stream, rimarrà sempre invariata e uguale. Tale nastro si accende solamente quando legge l'operazione finale come (count), altrimenti non fa niente.

L'interfaccia collection (che è quella che contiene tutti i metodi che riguardano strutture dati, quindi contains(), ecc..), chiama un metodo stream(), che restituisce uno stream< T >. Qualsiasi struttura dati si abbia, si può chiamare .stream(), e apre il nastro trasportatore per gli elementi di quella determinata struttura dati.

Le operazioni intermedie negli stream, quindi quelle fatte sul nastro, possono essere di 2 tipi:

-Stateless: Senza stato, come filter() o map(). Questo significa che ogni oggetto viene elaborato in modo indipendente dagli altri, quindi il risultato non dipende dall'insieme, ma dal singolo dato.

-Stateful: Con stato, come reduce() o max(). Questo invece significa che il risultato finale dipende da tutto l'insieme dei dati passati sullo stream, e quindi ricordarsi tutti gli elementi che vengono passati sul nastro, perché ad esempio per trovare il massimo tra tutti gli elementi tramite max(), non puoi ricordarti solo 1 elemento.

I METODI->

- Quando dobbiamo creare una lista in java, di solito con la libreria di java quella importata per gli ArrayList, usiamo quelle che sono le funzioni come add() e remove(). Ma per renderci la vita più semplice possiamo usare:

```
LinkedList<String> lista = List.of("Nobita", "Doraemon", "Suneo");
```

of(), è un metodo statico di list che ritorna una lista non modificabile, ma possiamo però scrivere la lista tutta in una volta come vediamo in tale foto. Questo serve per creare il magazzino dati che poi analizzeremo con gli stream. La parola magica che permette di applicare poi gli stream alla lista sarà .stream(), quindi nel caso di questa lista noi dovremmo chiamare lista.stream().

- Il predicato filter(Predicate p) è una condizione booleana sugli elementi dello stream. Ad esempio quando chiamiamo filter ci ritorna uno stream contenente gli elementi che soddisfano il predicato. Predicate è un'interfaccia funzionale che definisce il metodo test(Object o), prende in ingresso un Object e restituisce un boolean.

```
Predicate<Integer> positive = x -> x>0;
```

Come agisce il filter(Predicate p) : Il filtro prende la tua lambda (la condizione). Se

la lambda restituisce `true`, l'elemento passa al blocco successivo dello stream. Se restituisce `false`, l'elemento viene scartato e scompare dal flusso.

-`count()`, è un metodo che conta gli elementi di uno stream, ed è terminale. Prende lo stream filtrato e conta quanti elementi ci sono dentro. Restituisce una variabile primitiva di tipo `long`.

```
List<int> numeri = List.of(20,10,30,50,40);  
long contaMaggioriTrenta = numeri.stream()  
    .filter(n -> n>30)  
    .count();
```

In questo codice abbiamo anche il `filter()` che è il filtro per filtrare gli elementi dello stream che sono `>30`. Dopo averli filtrati gli elementi rimanenti verranno contati. Dentro la filter c'è l'espressione lambda che abbiamo visto nell'argomento prima, che con la freccia ha a sinistra l'input e a destra quello che deve restituire. Come vediamo dal codice, prima si chiama `nomeArray.stream()` per creare lo stream(il nastro), e poi si applicano le operazioni che vogliamo.

-`filter()`, è il metodo che filtra gli oggetti dello stream in base a quello che stiamo cercando.

-`reduce()`, è un metodo importante degli stream, e serve per racchiudere tutti i valori dello stream in un unico valore. Il reduce prende in input tali dati:

```
lista.stream().reduce(0, (accum, v) -> accum + v);  
lista.stream().reduce(0, Integer::sum) // Stessa cosa
```

-0-> Valore dello stesso tipo dello stream, è un valore iniziale, e se il nastro è vuoto ci restituirà 0;

-(`accum, v`) -> `accum + v` (L'accumulatore): Questa espressione lambda prende in input, `accum` che è la variabile dove verranno sommati tutti gli elementi dello stream, e `v` che è l'elemento che sta passando sul nastro in quel momento. Dal lato destro della lambda, quindi come valore di ritorno avremo la somma di tutti gli elementi su `accum`.

Sotto abbiamo la stessa identica operazione ma scritta in un modo più elegante. Invece di scrivere la lambda `(a, b) -> a + b`, dici a Java: *"Usa il metodo `sum` che esiste già dentro la classe `Integer`"*. Fa esattamente la stessa cosa, ma è più pulito da leggere.

-`map()`, è un metodo anch'esso molto importante, che applica una funzione ad ogni elemento dello stream che vi passa sotto. Quindi se la funzione fosse moltiplica per 2 ritornerebbe esattamente lo stesso numero di elementi che c'erano nello stream ma con un valore diverso.

```
List<Integer> lista = List.of(3,7,10);  
lista.stream().map(x -> x*2) // nuovo stream contiene 6,14,20
```

Quindi qui abbiamo il metodo `map` che viene chiamato e ogni numero dello stream viene moltiplicato per 2. Dentro la funzione `map` viene sempre passata un'espressione lambda.

-`collect()`, è un metodo diverso da quelli presentati fino ad adesso, perché quando dopo aver filtrato lo stream e trasformato, si vogliono rimettere questi dati di nuovo in una lista per poi riutilizzarli nel programma. `collect()` fa proprio questo, ossia raccoglie lo stream e lo chiude in una scatola.

```

List<int> numeriList = numeri.stream()
                                .collect(Collectors.toList());

// OPPURE
List<int> numeriList = numeri.stream()
                                .toList();

```

Questi 2 metodi con collect fanno la stessa cosa. Quello sotto è dopo java 16, dato che scrivere Collectors.toList() era troppo lungo hanno inserito solo il metodo toList().

Analizzando la prima riga di codice, collectors è il raccoglitore.

C'è una grande differenza tra quello che restituiscono le righe di codice. La prima con Collectors.toList() restituiscono una lista normale che è modificabile. La seconda riga con solo toList() restituisce un ArrayList immutabile.

-**comparator()**, non è un metodo dello stream, ma un aiutante. Serve a spiegare a java come confrontare gli oggetti complessi, ossia se hai una lista di elementi di tipo Persona, java non sa chi è maggiore o minore, ma tramite Comparator.comparing(Persona::getEta) gli stai dicendo di usare l'età per i confronti.

-**max()**, invece scorre tutto il nastro e trova l'elemento più grande in base al comparator che gli hai passato:

```

Optional<Persona> maxNumber = numeri.stream()
                                        .max(Comparator.comparing(Persona::getEta));

```

La parola Optional nella dichiarazione della variabile previene un fallimento perché se la lista dentro è vuota almeno non restituisce il NullPointerException.

-**distinct()**, operazione stateful, e che quindi deve ricordarsi di tutti gli elementi sullo stream, che restituisce uno stream con tutti elementi distinti, perché ad esempio se deve controllare se il 30 è già nello stream deve ricordarsi degli altri elementi;

-**sorted()**, anch'essa operazione stateful, che restituisce uno stream con elementi ordinati in base al comparator passato.

-**foreach()**, è un'operazione alternativa a collect() o toList(). Invece di salvare i dati in una lista, esegue un'azione immediata su ogni elemento (di solito stamparli a schermo).

```

chiavi.stream()
    .map(Chiave::getValore)
    .distinct()
    .sorted(Comparator.comparing(Chiave::getValore))
    .toList();

```

-**iterate()**, è un metodo stream che potrebbe generare stream infiniti, ma si pone un limite tramite la funzione limit(). Significa che:

```

Stream.iterate(2, n -> n*2)
    .limit(4)
    .forEach(System.out::println)

```

OUTPUT:

2 4 8 16

nella parte sx dell'espressione lambda viene messo il seme iniziale da cui partire e l'elemento n che passa nel nastro. A destra viene messa l'operazione con la quale andiamo a creare il nostro stream. Questo stream potrebbe andare avanti all'infinito fino ad esaurire la ram, ma c'è la funzione `limit()`, che all'interno accetta il numero di elementi massimo che quello stream può avere. Infine abbiamo il `foreach()` che ricordiamo esegue una determinata azione messa tra le sue parentesi per ogni elemento sullo stream, quindi in questo caso li stampa.

-`generate()`, rispetto ad `iterate()` crea elementi alla cieca per riempire lo stream, senza sapere nulla di cosa abbia prodotto qualche secondo prima. All'interno c'è una funzione `supplier` che è la funzione che fa stampare questi valori casuali.

```
Stream.generate(()->Math.round(Math.random()*10))
    .limit(5)
    .forEach(System.out::println);
```

In questo caso in questo codice genera e stampa 5 numeri casuali da 1 a 10. Il `supplier` in questo caso è `math.round(Math.random()*10)`.

-`peek()`, è un metodo simile a `foreach()` ma più funzionale, perché se ad esempio hai una catena di metodi di stream lunghi e c'è un errore e vuoi vedere da quale metodo viene questo errore, se lo fai con il `foreach` ammazzi lo stream, perché il `foreach()` termina lo stream. Invece con il `peek` prende un elemento specifico dal nastro, lo stampa e lo rimette al suo posto.

Java

```
List<Integer> numeri = List.of(1, 2, 3, 4, 5);

List<Integer> elaborati = numeri.stream()
    .peek(n -> System.out.println("Partenza: " + n)) // Sbircio il nastro all'inizio
    .filter(n -> n % 2 == 0) // Tengo solo i pari
    .peek(n -> System.out.println("Sopravvissuto al filtro: " + n)) // Sbircio chi è passato
    .map(n -> n * 10)
    .toList();
```

PROGRAMMAZIONE PARALLELA->

Dato che i processori hanno vari core, si devono sfruttare tutti, e questo va fatto con la programmazione parallela. Il trucco della programmazione parallela sta sul comando `parallelStream()`. Quando per la programmazione sequenziale con gli stream usiamo il comando `stream()`, analizziamo gli elementi dello stream uno alla volta. Con il comando `parallelStream()` al posto di `stream()` dividiamo in blocchi gli elementi dello stream in base a quanti sono i core della nostra cpu, e ne assegniamo uno ad ognuno così che possano lavorare tutti in parallelo.

```
long c = nomi.parallelStream()
    .map(String::toUpperCase)
    .filter(s -> s.equals("NOBI"))
    .count();
```

La programmazione parallela conviene sugli `arrayList` ma non sulle `linkedList`. Questo perché l'`arrayList` già sappiamo quanto è grande perché è conservato in memoria contigua e

divide meglio gli elementi per ogni core. Invece con le linkedList gli elementi sono sparsi in memoria e quindi si dovrebbe scorrere tutta la lista per trovarli e poi dividerli.

Il comando `findAny()` è un metodo che applica lo short-circuiting, cioè tra una lista di elementi, appena trova il primo che rispetta quella condizione lo stream si bloccherà.

```
Optional<Persona> r = team.stream()
    .filter(p -> p.isRuolo("Programmer"))
    .findAny();

if (r.isPresent()) System.out.println(r.get().getNome());
```

INTSTREAM->

Sono gli stream per lavorare sugli oggetti `int` (quindi oggetti e numeri interi). Si sono creati per una questione di velocità del processore dato che quando facciamo `Stream< Integer >` dobbiamo convertire tutti i numeri primitivi `int` in `integer`, e questo rallenta molto il processore. dato che gli `intstream` lavorano unicamente su numeri interi primitivi, java gli ha donato dei metodi matematici puri, come `sum()`, `average()`, `min()`, `max()`.

-`rangeClosed(1,3)`: genera letteralmente lo stream `[1,2,3]`. Se invece fosse stato solo `range(1,3)`, allora avrebbe escluso il 3.

-`sum()`: come capiamo subito somma i valori dello stream, e non c'è bisogno di fare `Integer::sum`.

```
int somma = IntStream.rangeClosed(1,3).sum(); // somma = 1+2+3 = 6
```

-`mapToInt()`, è quella funzione che fa passare gli elementi dello stream dal mondo degli oggetti al mondo dei primitivi.

```
int result = Stream.of("ciao", "sono", "simone")
    .mapToInt(x->x.length())
    .sum();
```

Come vediamo inizialmente crea uno stream con 3 parole. Poi prende la lunghezza di queste stringhe tramite l'espressione lambda che vediamo dentro `mapToInt` che quindi converte le stringhe a numeri interi. Dopo questo si può tranquillamente fare la somma.

-`boxed()`, fa il contrario di `mapToInt`, ossia che partendo da uno stream di oggetti primitivi `intStream`, passa a `Stream< integer >`, perché per poter ricreare poi le liste con gli elementi trasformati tramite `toList()` bisogna per forza passare al mondo degli oggetti.

```
Stream<Integer> s = IntStream.rangeClosed(1,10)
    .boxed();
```

-`mapToObject()`: è un'alternativa elegantissima ai vecchi cicli `for`.

```
Java

IntStream.rangeClosed(0,3)
    .mapToObj(i -> lista.get(i));
```

Sta creando un nastro di indici `[0, 1, 2, 3]`. Poi usa ogni indice `i` per andare a

"pescare" un elemento da una lista esterna. Il nastro si trasforma quindi da un flusso di numeri a un flusso di oggetti estratti da quella lista.

INTERFACCIA FUNCTION->

L'interfaccia funzionale `Function<T,R>` definisce un metodo chiamato `apply()` che prende in ingresso un oggetto di tipo generico `T` e ritorna un tipo generico `R`. Per esempio:

```
Function<String, Integer> stringLength = x -> x.length();
```

Qui ha un oggetto di tipo `string` all'ingresso, e in uscita ha un oggetto di tipo `integer` che indica la lunghezza di tale stringa.

```
int result = Stream.of("truth", "flows", "to", "them", "sweetly", "by", "nature")
    .map(stringLength)
    .reduce(0, Integer::sum);

System.out.println(result);
```

OUTPUT:
31

Qui vediamo `map` prendere in input la funzione `stringLength` definita sopra per calcolare la lunghezza di ogni parola e quindi `reduce()` per sommarle partendo da 0. Sottolineiamo il fatto che non dobbiamo vedere `stringLength` (la creiamo noi) come se fosse un valore ma una vera e propria funzione, che quindi definisce un comportamento che può essere passato a `map()`.

INTERFACCIA SUPPLIER->

Un `supplier< T >` è un'interfaccia funzionale che applica un metodo `get()` che prende in input niente, e in output restituisce un oggetto di tipo `T`.

```
Supplier<String> sup = () -> "Ciao ciao";
String s = sup.get();
```

OUTPUT:
Ciao ciao

Quindi richiama poi il `get` che ci dà la stringa messa in output nell'espressione lambda.

ESEMPIO su come l'interfaccia `supplier` con java 8 sia più funzionale rispetto ai design pattern:

```
public class Creator {
    public static Prodotto getProdotto(String name) {
        switch (name) {
            case "primo": return new ProdottoA();
            case "secondo": return new ProdottoB();
            case "terzo": return new ProdottoC();
            case "quarto": return new ProdottoD();
            default: return new ProdottoA();
        }
    }
}
```

```
Prodotto p1 = Creator.getProdotto("primo");
```

Questo è il primo blocco di codice con uno switch infinito, e difficile da aggiornare in futuro se si volessero mettere altri prodotti.

Con l'interfaccia supplier invece funziona proprio come un costruttore, ossia non prende niente in ingresso, e sputa fuori un oggetto:

Se usassimo un Supplier definito come

```
Supplier<Prodotto> prodSupplier = ProdottoA::new;
```

La linea di codice per creare p1 sarebbe tradotta in questo modo

```
Prodotto p1 = prodSupplier.get();
```

L'oggetto viene sputato fuori tramite il metodo get.

```
Map<String, Supplier<Prodotto>> map = Map.of("primo", ProdottoA::new, "secondo",  
ProdottoB::new, "terzo", ProdottoC::new);
```

Poi abbiamo una mappa per la creazione di tali prodotti, dove mettiamo prima String("primo" "secondo" ecc..) e dopo la creazione del prodotto con prodottoA::new.

```
public static Prodotto getProdotto(String name) {  
    Supplier<Prodotto> s = map.get(name);  
    if (s != null) return s.get();  
    return new ProdottoA(); // Default  
}
```

Infine qui abbiamo la funzione per farci restituire il prodotto. Creiamo un oggetto s per avere il nome del prodotto, in base al nome che è la stringa primo, secondo, se c'è ce lo ridà, altrimenti ci restituisce per default il prodotto A. Quindi è una struttura come lo switch ma molto più efficiente.